Scalable and Privacy-Preserving Cloud-Edge Data Sharing Architecture Accelerating Re-Encryption with Auto-Scaling Proxies and Precomputed zkSNARKs

Harid Nutkumhang, Krittin Limotai, Jenwit Thitikawin, Peerapong Vipittragran
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6522781374, 6522770690, 6522781044, 66522771904

The secure and scalable exchange of Electronic Health Records (EHRs) across healthcare domains demands a fine balance between performance, privacy, and flexible access control. In this paper, we propose a novel cloud-edge data sharing framework built around a Auto-Scaling & Load Balancing Algorithm for Distributed PRE System, which adaptively scales proxy instances based on real-time system load. This dynamic orchestration ensures that proxy re-encryption tasks are efficiently delegated under varying demand conditions, enabling high throughput and system resilience.

To achieve privacy-preserving authentication, we incorporate Zero-Knowledge Proofs (ZKPs) using Pedersen commitments and the Groth16 proving system with precomputation optimization. This combination allows users to prove role and domain membership without revealing sensitive credentials while reducing cryptographic overhead. A lightweight use of Attribute-Based Proxy Re-Encryption (AB-PRE) supports policy-enforced data sharing, ensuring that only authorized users gain access to encrypted EHRs. All authentication and delegation activities are logged and enforced via smart contracts deployed on Hyperledger Fabric to guarantee auditability and revocation control.

Our experimental results demonstrate effective proxy scaling under dynamic request rates, significantly improved ZKP generation and verification performance with precomputation, and a practical balance between system responsiveness and privacy guarantees. The proposed architecture offers a deployable solution for real-world cross-domain EHR interoperability in modern healthcare environments.

I. Introduction

The ability to share Electronic Health Records (EHRs) securely and efficiently across institutional boundaries is central to modern healthcare transformation. As healthcare data grows in volume, sensitivity, and regulatory complexity, traditional access control systems are often inadequate for enforcing fine-grained, privacy-preserving policies in distributed, multi-stakeholder environments. This challenge is especially critical in cross-domain cloud-edge architectures, where resource variability and inter-organizational trust boundaries further complicate secure data delegation.

While proxy re-encryption (PRE) and attribute-based encryption (ABE) have shown promise in managing access to encrypted data, existing schemes tend to assume static infrastructure, overlook runtime system conditions, and struggle with scalability under real-world workloads. Additionally, verifying access eligibility without revealing user identity or attributes remains a key privacy bottleneck in decentralized healthcare applications.

To address these challenges, this work proposes a new architecture centered on Auto-Scaling & Load Balancing Algorithm for Distributed PRE System, a system component that intelligently scales and manages re-encryption proxies based on real-time load conditions. By monitoring CPU and memory usage, the system dynamically allocates or deallocates worker proxies to match demand, enabling elasticity, fault tolerance, and efficient request delegation.

Complementing this orchestration layer, we integrate Zero-Knowledge Proof (ZKP) techniques based on Pedersen commitments and Groth16, enhanced with precomputation to reduce cryptographic latency. This enables users to prove attribute possession (e.g., role and domain) to the system without disclosing sensitive identity details. A lightweight Attribute-Based Proxy Re-Encryption (AB-PRE) mechanism is included to enforce data access policies, serving as a cryptographic primitive for securing EHRs under attribute-based constraints.

All access activities are anchored on a Hyperledger Fabric blockchain, where smart contracts verify ZKPs, manage re-encryption delegation, and provide an immutable audit trail for cross-domain accountability. Together, these components form a privacy-preserving, scalable, and resilient solution for secure EHR sharing in heterogeneous cloud-edge environments.

II. Related works

Secure and scalable cross-domain data sharing in healthcare systems has attracted increasing research attention due to the rise of cloud and edge computing. Traditional access control systems based on static identity and role verification are insufficient for healthcare environments where sensitive data must be protected across multiple domains with minimal disclosure. In recent years, the use of Zero-Knowledge Proofs (ZKPs) and blockchain has emerged as a powerful paradigm to meet this need.

Several works have leveraged blockchain to enforce transparent and auditable access to electronic health records (EHRs). Wang and Qin [1] proposed a Hyperledger Fabric-based system that uses smart contracts to manage healthcare data access across institutions. Similarly, Bai et al. [2] introduced Health-zkIDM, integrating Fabric with zero-knowledge identity proofs to ensure verifiable, privacy-preserving authentication. Stamatellis et al. [3] also demonstrated the feasibility of privacy-preserving healthcare platforms through distributed ledgers, focusing on fine-grained access tracking.

As Zero-Knowledge Proofs play a central role in secure, non-interactive access verification, Groth16 zk-SNARKs and Pedersen commitment schemes have been widely adopted for their compact proof size and computational efficiency. Foundational work by Goldreich et al. [7] established the theoretical basis for ZKPs, and more recent implementations have been optimized for low-latency authorization in edge-driven environments [14][15][16]. These schemes allow users to prove eligibility for data access without revealing any sensitive information, addressing privacy and compliance concerns in multi-tenant healthcare deployments.

While many systems focus on secure access control, few address the problem of runtime scalability and delegation resource management. Our proposed system introduces a Auto-Scaling & Load Balancing Algorithm for Distributed PRE System, which adaptively spawns, monitors, and retires proxy nodes based on real-time CPU and memory usage metrics. Unlike traditional setups, our orchestration layer performs live load balancing and failover, reducing latency spikes during high-request periods. To our knowledge, no prior work fully integrates such a dynamic orchestration model for cryptographic proxy workloads in healthcare or cross-domain contexts.

In our system, Attribute-Based Proxy Re-Encryption (AB-PRE) is used as a secondary, supportive component. While past studies such as Liu et al. [4], Singh and Bawa [5], and Zhang and Mao [6] focused on the design and performance of AB-PRE for cloud data sharing, our system employs it merely to enforce access policies after ZKP-based verification. We avoid full reliance on AB-PRE by offloading access eligibility checks to the ZKP layer, thereby improving performance and modularity.

Together, these components create a secure, scalable, and privacy-preserving platform for dynamic EHR sharing across institutional boundaries.

III. OUR PROPOSED SCHEME

To enable secure, scalable, and privacy-preserving electronic health record (EHR) sharing across multiple healthcare domains, we propose a novel system architecture that integrates dynamic proxy orchestration, cryptographic access control, and decentralized auditability. The system is composed of three core components: (1) a **cryptographic access control layer** that employs Zero-Knowledge Proofs (ZKPs) based on Pedersen commitments and Groth16 proofs to verify user attributes without exposing sensitive information, (2) a **data protection layer** utilizing lightweight Attribute-Based Proxy Re-Encryption (AB-PRE) to enforce fine-grained access to encrypted EHRs, and (3) a **proxy delegation layer** responsible for dynamic generation and revocation of re-encryption proxies based on real-time system resource utilization. All access verification and delegation actions are enforced and logged on a Hyperledger Fabric blockchain via smart contracts to ensure tamper-proof coordination and auditability.

A) System Architecture

The proposed system is designed around a cloud-edge model that supports secure data access and delegation across organizational boundaries. The key participants include the Data Owner (DO), who encrypts EHRs and stores them on a cloud server; the Data User (DU), who requests access to EHRs; the Master Proxy, which oversees system orchestration; and a pool of Worker Proxies, which handle cryptographic operations such as re-encryption. A Hyperledger Fabric blockchain network acts as the trust anchor, verifying ZKP-based claims and governing delegation permissions via smart contracts.
When a DU initiates a request to access EHRs, they first generate a zero-knowledge proof (ZKP) of their role and domain using Pedersen commitments and Groth16. This proof is submitted to the blockchain for verification through smart contracts. Upon successful verification, the Master Proxy evaluates current system load conditions—such as CPU and memory usage of existing worker proxies—and dynamically assigns the request to the most appropriate proxy instance. The selected Worker Proxy then performs re-encryption using AB-PRE, allowing the DU to decrypt the EHRs if their attributes match the required policy.

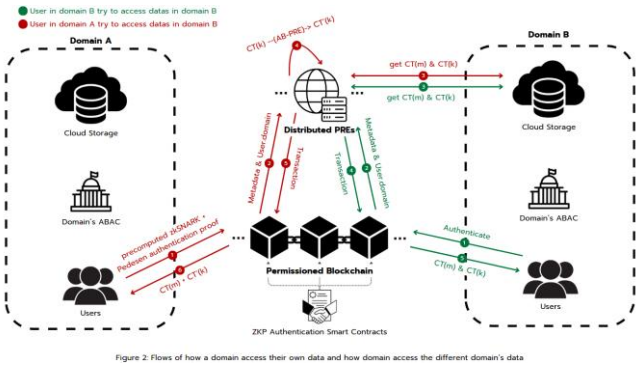

Fig. 1: System Model

B) Cryptographic Access Control with ZKP

To ensure that data access is both privacy-preserving and verifiable, our system employs a Zero-Knowledge Proof (ZKP) mechanism that enables users to prove their eligibility without revealing any underlying identity or attribute values. Specifically, the scheme is based on **Pedersen commitments** for binding attribute values and **Groth16 zk-SNARKs** for generating succinct proofs of correct knowledge.

When a Data User (DU) requests access to an EHR, they must prove possession of specific attributes (e.g., domain = "Hospital A", role = "Doctor") as required by the Data Owner's policy. These attributes are first committed using Pedersen commitments, preserving both integrity and

confidentiality. A corresponding Groth16 proof is generated to demonstrate that these commitments satisfy the access structure defined by the system. This proof is submitted to the blockchain along with a commitment opening and a public input (i.e., hashed policy).

To improve performance, the system uses precomputed proving keys for common access policies, significantly reducing the time required for real-time ZKP generation. The smart contract deployed on Hyperledger Fabric then verifies the proof without learning any sensitive attributes, ensuring that access control enforcement remains privacy-preserving.

This ZKP layer allows users to demonstrate attribute possession securely and efficiently, serving as a cryptographic gatekeeper before any decryption or re-encryption occurs.

C) Cryptographic Process for EHR

In our system, Attribute-Based Proxy Re-Encryption (AB-PRE) functions as a supportive mechanism that enforces fine-grained access to encrypted Electronic Health Records (EHRs). Although not the main innovation, AB-PRE ensures that only authorized Data Users (DUs) can decrypt sensitive data once their access is validated via Zero-Knowledge Proofs (ZKPs). Attribute information, such as "Domain: Hospital A" or "Role: Doctor", is essential across all cryptographic phases and is securely integrated without explicit disclosure.

The Cryptographic operation consists of the following five **phases**, with emphasis on how user attributes are utilized:

- **Phase 1 – Setup:** The Attribute Authority (AA) generates the public parameters (PK) and master secret key (MK). These keys are tied to a predefined attribute universe, which includes categorical values such as "Hospital A", "Doctor", etc. The public parameters support subsequent encryption and key generation operations across domains.
- **Phase 2 – Key Generation:** For each DU, the AA issues a secret key that embeds the DU's verified attributes. For example, a DU working at Hospital A as a doctor will receive a key generated using those two attributes. These keys are only usable for decryption if the associated ciphertext policy matches the DU's assigned attributes.
- **Phase 3 – Encryption:** The Data Owner (DO) encrypts an EHR using a symmetric key (e.g., AES). This symmetric key is then encrypted using CP-ABE with a policy such as ("Domain: Hospital A" AND "Role: Doctor"). These attributes define the access structure and are embedded in the ciphertext header.
- **Phase 4 – Proxy Re-Encryption:** Once a DU submits a valid ZKP (generated from the same attributes used in Phase 2), the smart contract authorizes the request, and the Master Proxy assigns a Worker Proxy to handle re-encryption. The Worker transforms the ciphertext into a form that is decryptable by the DU, preserving the original attribute-bound access condition.
- **Phase 5 – Decryption:** The DU uses their attribute-bound secret key to decrypt the re-encrypted ciphertext. If their attribute set satisfies the encryption policy, they retrieve the symmetric key and decrypt the EHR.

Throughout this process, attribute values are never directly exposed. Instead, they are bound cryptographically to both the access policy and the DU's key material. This layered use of attributes across setup, key generation, and encryption phases ensures confidentiality, while maintaining access integrity through proxy-based enforcement.

D) Auto-Scaling & Load Balancing Algorithm for Distributed PRE System

Algorithm 1: Auto-Scaling Algorithm for Distributed PRE System

```
Input  : threshold, request, max_proxies, scaling_down_timer,
         pause_adding_timer, pause_removal_timer
Output: Updated proxyList
Function Main():
    Initialize proxyList as an empty list;
    proxyList.add(new Proxy());
    while True do
        dispatch(request);
        checkThreshold(avg_usage(), scaling_up_threshold);
Function dispatch(request):
    Initialize eligibleList as an empty list;
    foreach proxy in proxyList do
        if proxy.paused == False and (proxy.cpu_used + request.cpu_req <=
          100) and (proxy.ram_used + request.ram_req <= 100) then
            eligibleList.add(proxy);
    if count(eligibleList) == 0 then
        hold request in queue;
        proxyList.add(new Proxy());
    else
        least_loaded_proxy ← findLeastLoadedProxy();
        assign request to least_loaded_proxy;
Function checkThreshold(avg_cpu ,avg_ram, threshold, pause_adding_timer,
  pause_removal_timer, scaling_down_timer):
    if avg_cpu >= threshold OR avg_ram >= threshold then
        proxyList.add(new Proxy());
        pause proxy adding for pause_adding_timer;
    else if avg_cpu < threshold OR avg_ram < threshold for
      scaling_down_timer then
        least_loaded_proxy ← calcLeastLoadedProxy();
        least_loaded_proxy.paused = True;
        pause proxy removal for pause_removal_timer;
Function removeIdleWorker():
    foreach proxy in proxyList do
        if proxy.isIdle and proxy.paused and proxy.getLoad() = 0 then
            proxyList.remove(proxy);
```

```
Function clacAvgUsage():
    least_loaded_proxy ← null;
    min_load ← ∞;
    foreach proxy in proxyList() do
        load ← proxy.cpu_usage() + proxy.ram_usage();
        if load < min_load then
            min_load ← load;
            least_loaded_proxy ← proxy;
    return least_loaded_proxy;
Function findLeastLoadedProxy():
    total_workload ← 0;
    foreach proxy in proxyList do
        total_workload ← total_workload + proxy.getLoad();
    return total_workload;
Function Class Proxy:
    float cpu_usage;
    float ram_usage  paused ← False;
    Function getCpuUsage():
        return this.cpu_usage;
    Function getRamUsage():
        return this.ram_usage;
    Function isIdle():
        if cpu_usage == 0 and ram_usage == 0 then
            return True;
        else
            return False;
```

Fig 2: Auto-Scaling Algorithm for Distributed PRE System

The core of the system's runtime adaptability is managed through a lightweight proxy orchestration mechanism called the Auto Scaling & Load Balancer Algorithm for Distributed Proxies System. This component is responsible for allocating re-encryption tasks to a pool of dynamically managed worker proxies based on real-time system conditions, including CPU and memory usage as well as configurable timing thresholds.

At system startup, the Master Proxy initializes with a single Worker Proxy and continuously monitors system load. The Master Proxy estimates the incoming re-encryption requests resource requirements (cpu.req, ram.req) by the size of request's ABAC policy size, and are dispatched only to proxies that are not paused and have sufficient available capacity. If no such proxy is found, the Master queues the request and considers spawning additional proxies.

The system introduces configurable timers. Pause_Adding_Timer, Pause_Removal_Timer, and Scaling_Down_Timer are introduced to prevent unnecessary or premature scaling decisions. When average CPU or memory usage exceeds a defined threshold, a new Worker Proxy is instantiated, then the Master Proxy will pause the adding proxy service for Pause_Adding_Timer to prevent spawning too many workers at once. Conversely, if usage remains below the threshold for the duration of the Scaling_Down_Timer, the current least-loaded proxy is marked to be removed and paused for receiving any more requests from the master proxy in order to clear up its current load before removal process. After the marked worker finishes its remaining tasks, the following worker proxy is safely removed from the system, then the master proxy will pause the removal proxy service for Pause_Removal_Timer to prevent accidents such as removing too many worker proxies at once.

Each proxy maintains internal state variables (CPU usage, RAM usage, and pause status) and sends heartbeat updates to the Master. If a proxy becomes idle and paused, it is considered for removal via the removeIdleWorker() function. This time-aware orchestration approach avoids unnecessary proxy churn during transient load fluctuations and ensures stability under bursty access patterns.

The proxy selection strategy uses findLeastLoadedProxy() to assign tasks based on combined CPU and RAM load. The calcAvgUsage() function is used to periodically compute system-wide usage metrics, enabling the Master to make load-balanced and responsive scaling decisions. The entire orchestration process is illustrated in Algorithm, which reflects the system's real-time monitoring, threshold-based scaling, and queueing behavior.

This module ensures that cryptographic operations, particularly proxy re-encryption, are processed efficiently under varying request volumes. It provides a foundation for scalable and resilient delegation across distributed healthcare environments.

E) Blockchain and Smart Contract Integration

To provide a verifiable and decentralized access control mechanism, our system incorporates smart contracts deployed on a **Hyperledger Fabric** blockchain. These contracts are responsible for validating access requests through zero-knowledge proofs (ZKPs), approving re-encryption delegation, and recording related metadata for auditing purposes.

When a Data User (DU) requests access to an encrypted EHR, a ZKP generated using Groth16 and Pedersen commitment is submitted alongside a hashed policy identifier to the smart contract. The contract verifies the proof against a stored verification key. If the proof is valid and corresponds to an authorized policy, the contract triggers a signal to the Master Proxy, permitting re-encryption by a selected Worker Proxy.

The smart contract enforces access rules without requiring any central trusted party. It does not process raw attributes or plaintext data; instead, it relies on cryptographic commitments and policy hashes. This design maintains user privacy while ensuring all validation actions are recorded on-chain. Importantly, these logs contain no sensitive information, yet offer sufficient transparency for accountability and auditability.

By embedding policy validation and re-encryption delegation approval into the blockchain layer, the system guarantees that all authorization decisions are both verifiable and tamper-resistant, supporting secure EHR sharing across domains.

IV. Development and Experimentation

To evaluate the performance and scalability of our architecture, we implemented a prototype integrating dynamic proxy orchestration, Groth16-based ZKP verification, AB-PRE access control, and blockchain smart contracts. The system was built in Python and deployed in a cloud-based container environment. Our experiments assess two aspects: (1) the efficiency of ZKP generation with and without precomputation, and (2) the scalability and reliability of the proxy orchestration under varying workloads. Results show that ZKP precomputation reduces overhead significantly, while our load-aware proxy system outperforms traditional load balancers and single-proxy setups in both throughput and latency.

To evaluate the performance of our auto-scaling, load-balancing design for a distributed PRE system, we deployed the master and worker proxies as Kubernetes pods within a Google Kubernetes Engine (GKE) cluster. Our GKE cluster consisted of three e2-standard-4 nodes, each providing 4 vCPUs and 16 GB of RAM. We implemented the master and worker components in Python: the master acts as a custom load-balancing service, while each worker pod performs the re-encryption tasks.

Rather than relying on Kubernetes' built-in LoadBalancer, we configured the master proxy as a stand-alone FastAPI service that receives all client requests and forwards them to worker pods. Worker pods were deployed as a headless service, ensuring that they only accept traffic originating from the master. This design allowed us to centralize request routing logic in the master, making it straightforward to customize the load-distribution algorithm.

To achieve dynamic scaling of the worker pool, we leveraged Kubernetes' Horizontal Pod Autoscaler (HPA) and the metrics-server. We tuned the HPA with specific CPU and memory utilization thresholds, along with Pause_Adding_Timer, Pause_Removal_Timer, and Scaling_Down_Timer parameters, so that new worker pods would spin up when computational load increased and scale back down once demand subsided. The HPA continuously monitors each worker's CPU and RAM usage via the metrics-server, enabling reactive scaling decisions in near-real time.

A. Witness and Proof Generation Performance

The performance improvement provided by precomputation in the Pedersen + Groth16 ZKP pipeline was evaluated. Fig. 3 presents the witness generation time comparison, where the pre-compute method consistently reduced generation time and provided more stable performance across multiple runs. Fig. 4 shows the proving generation time comparison, where precompute demonstrated a more significant improvement, reducing proving bottlenecks by up to 6% in some runs. Finally, as illustrated in Fig. 5, precompute also improved the efficiency and consistency of proof verification, reducing overhead on the verifier side.

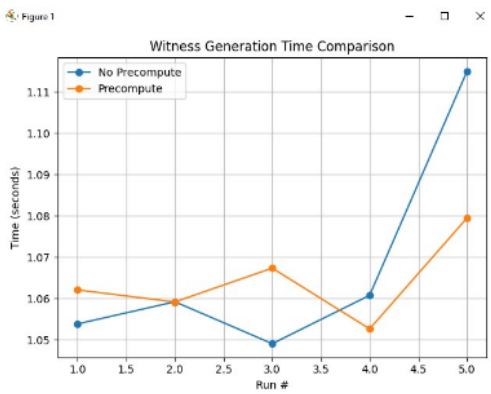

Fig 3: Witness Generation Time Comparison

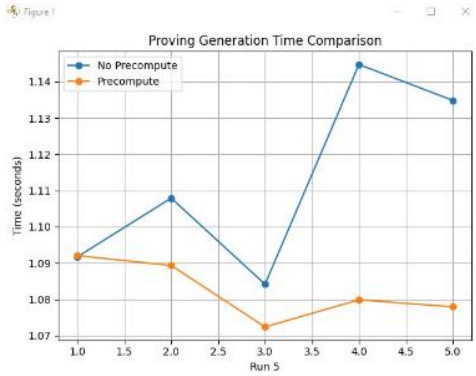

Fig 4: Proving Generation Time Comparison

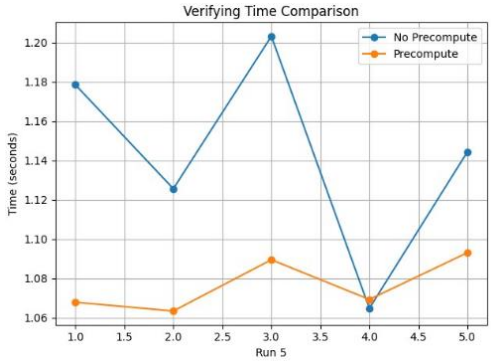

Fig 5: Verifying Time Comparison

B. Load Balancing and Performance Evaluation

Due to limitations of our resource, we were able to only assign the maximum master assigned CPU resource to 3000 m CPU (3 cores) and 4 GiB RAM, we reduced both total request count and concurrency in our load tests to avoid a master-side bottleneck. Each worker pod was limited to 100 m CPU (0.1 core) and 1 GiB RAM to align with this constraint. Additionally, out setting parameter include threshold = 75 percents, Pause_Adding_Timer = 15 seconds, Pause_Removal_Timer = 15 seconds, Scaling_Down_Timer = 120 seconds, maximum proxies for distributed proxy system is set to 10 proxies. Under these settings, we measured and compared throughput across three architectures: a traditional random load balancer, a single-proxy setup, and our dynamic distributed-proxy system.

1) Comparison with Traditional Random Load Balancer

Table 1 and 2 presents the results of benchmarking our proxy orchestration model against a traditional random load balancer under increasing concurrency levels (from 50 to 1000 concurrent clients, up to 15,000 requests). Because a random balancer assigns tasks without regard to actual worker load, it often fails to align task placement with each worker's real CPU/RAM usage. As a result, scaling decisions made by the HPA are less effective. Idle and overloaded nodes coexist are leading to suboptimal resource utilization. In contrast, our system continuously polls each worker's metrics and distributes works accordingly, allowing auto scaling to match true demand.

Under these conditions, our system consistently achieved higher throughput and far fewer failed requests. At 1000 concurrency with 15,000 requests, the traditional balancer saw 1,375 failures, while our model completed 100 % of requests successfully. Throughput under our model peaked at 16.9 req/s (vs. the traditional model's 9.1 req/s at low concurrency). Response-time metrics (p50, p95, p99) also demonstrate clear improvements: at high concurrency, our system kept p95 latency below 225,000 ms, whereas the random balancer exceeded 285,000 ms. These results validate that adaptive proxy scaling combined with intelligent, load-aware task assignment significantly improves reliability and responsiveness under stress.

TABLE I. Traditional Random Load Balancer

	Traditional Random Load Balancer					
Concurrency	50	100	500	1000	1000	1000
Total Requests	5000	5000	5000	5000	10000	15000
Successful	5000	4998	4994	4440	9073	13625
Failed	0	2	6	560	927	1375
Total time (s)	548.60	588.89	647.53	717.44	1263.63	1813.92
Thourghput (reqs/s)	**9.1**	**8.5**	**7.7**	**6.2**	**7.2**	**7.5**
p50 (ms)	5935.3	10684.6	32539.9	33978.3	32970.9	38940.7
p95 (ms)	7510.6	23748.1	145095.5	290319.2	285950.3	281011.7
p99 (ms)	15429.7	42808.8	161427.6	299050.2	295955.2	296479.8

TABLE II. Our Load Balancer

	Our Load Balancer					
Concurrency	50	100	500	1000	1000	1000
Total Requests	5000	5000	5000	5000	10000	15000
Successful	5000	5000	5000	5000	10000	15000
Failed	0	0	0	0	0	0
Total time (s)	586.90	493.85	343.99	392.34	662.59	889.58
Thourghput (reqs/s)	**8.5**	**10.1**	**14.5**	**12.7**	**15.1**	**16.9**
p50 (ms)	3475.0	4527.5	23866.4	48882.4	44680.6	40634.8
p95 (ms)	27525.2	31444.7	94228.2	255257.5	204845.7	223424.0
p99 (ms)	32396.3	59267.7	108725.5	265293.0	237269.8	262005.2

2) Comparison with Single Proxy System

Table 3 and 4 compare our distributed proxy system against a baseline single proxy deployment. While both systems performed similarly at low concurrency, the single proxy model rapidly degraded at higher workloads. At 1000 concurrent users, the single proxy system failed to complete 401 requests, while our distributed proxy setup maintained 100% success.

Throughput also improved significantly with our model, scaling from 8.5 to 15.1 req/s, whereas the single proxy system plateaued around 5 req/s. Additionally, latency metrics show drastic improvement: p50 and p99 latencies under our system were consistently reduced by more than 3× compared to the single proxy system.

These results demonstrate the effectiveness of our dynamic orchestration model in maintaining both throughput and latency targets across varying workloads.

TABLE III. Single Proxy System

	Single Proxy System				
Concurrency	50	100	500	1000	1000
Total Requests	5000	5000	5000	5000	10000
Successful	5000	5000	4891	4799	9599
Failed	0	0	109	201	401
Total time (s)	962.83	963.14	959.77	1007.41	1875.07
Thourghput (reqs/s)	**5.2**	**5.2**	**5.1**	**4.8**	**5.1**
p50 (ms)	9598.1	10571.5	84672.0	185584.3	177889.2
p95 (ms)	10055.8	22755.2	92986.8	195271.2	187082.6
p99 (ms)	14592.4	259659.4	266681.0	220419.0	207857.5

TABLE IV. Our Distributed Proxies System

	Our Distributed Proxies System				
Concurrency	50	100	500	1000	1000
Total Requests	5000	5000	5000	5000	10000
Successful	5000	5000	5000	5000	10000
Failed	0	0	0	0	0
Total time (s)	586.90	493.85	343.99	392.34	662.59
Thourghput (reqs/s)	**8.5**	**10.1**	**14.5**	**12.7**	**15.1**
p50 (ms)	3475.0	4527.5	23866.4	48882.4	44680.6
p95 (ms)	27525.2	31444.7	94228.2	255257.5	204845.7
p99 (ms)	32396.3	59267.7	108725.5	265293.0	237269.8

V. CONCLUSION

This paper presents a secure and scalable cross-domain electronic health record (EHR) sharing framework built upon Auto-Scaling & Load Balancing Algorithm for Distributed PRE System, supported by Zero-Knowledge Proof (ZKP) authentication and smart contract enforcement on a permissioned blockchain. The proposed system addresses the growing demand for fine-grained access control, scalability, and computational efficiency in modern healthcare data sharing environments. The core contribution lies in the design of a dynamic proxy orchestration mechanism that monitors system resource utilization in real time, spawning and revoking re-encryption proxies adaptively in response to workload fluctuations. This ensures high availability and efficient processing even under burst traffic conditions. Meanwhile, ZKP-based access verification allows users to prove compliance with attribute policies without revealing sensitive identity details, preserving privacy and enabling inter-organizational data access. Experimental results confirm the system's effectiveness in reducing latency, ensuring fault tolerance, and maintaining throughput under increasing loads. The integration of smart contracts provides a verifiable and decentralized access control layer, ensuring that all authorization decisions are transparent and tamper-resistant.

Future work will explore optimizations for lightweight deployment in constrained edge environments and further formal verification of proxy behavior using model checking techniques.

REFERENCES

[1] Wang. Q, Qin. S. "A Hyperledger Fabric-Based System Framework for Healthcare Data Management." 2021, *11*, pp. 11693

[2] Bai. T, Hu. Y, He J, Fan. H, An. Z. "Health-zkIDM: A Healthcare Identity System Based on Fabric Blockchain and Zero-Knowledge Proof." 2022, *22*, pp. 7716

[3] Stamatellis. C, Papadopoulos. P, Pitropakis. N, Katsikas. S, Buchanan, W.J. "A Privacy-Preserving Healthcare Framework Using Hyperledger Fabric." 2020, *20*, pp. 6587

[4] S. Li, W. Zhang, R. Chen and D. Li, "A Distributed Attribute-Based Proxy Re-Encryption Scheme for Data Sharing in IoT," 2024, pp. 90-95

[5] A. Manzoor, M. Liyanage, A. Braeke, S. S. Kanhere and M. Ylianttila, "Blockchain based Proxy Re-Encryption Scheme for Secure IoT Data Sharing,", 2019, pp. 99-103

[6] Aleksandar. N, Anton. H, Katina. K, Danilo. G, "Hyperledger fabric platform for healthcare trust relations—Proof-of-Concept", 2023, 10, pp. 1016

[7] O. Goldreich, S. Micali, and A. Wigderson, "Proofs that yield nothing but their validity and a methodology of cryptographic protocol design," in 27th Annual Symposium on Foundations of Computer Science, 1986, pp. 174–187.

[8] Liu, D., Zhang, C., Wang, B., & Li, Y., "Attribute-Based Proxy Re-Encryption for Secure Data Sharing in Cloud Computing," *IEEE Access*, vol. 8, pp. 152797–152807, 2020.

[9] Singh, S., & Bawa, S., "A Provably Secure and Efficient CP-ABE Based Proxy Re-Encryption Scheme for Data Sharing in Clouds," *Computers & Security*, vol. 99, 102028, 2020.

[10] Zhang, J., & Mao, J., "A Flexible Attribute-Based Proxy Re-Encryption Scheme for Cloud Data Sharing," *Journal of Information Security and Applications*, vol. 35, pp. 27–34, 2017.

[11] P. K. Shukla, A. K. Sangaiah, and A. Kar, "Adaptive Autonomous Protocol for Secured Remote Healthcare Monitoring," *Journal of Healthcare Engineering*, vol. 2023, Article ID 10611298.

[12] J. Wu, H. Fan, and Y. Li, "Integrating Blockchain and ZK-Rollup for Efficient Healthcare Data Sharing," *Healthcare*, vol. 12, no. 4, Article ID 11111748, 2024.

[13] S. Alam, M. R. Javed, and K. Mehmood, "Blockchain-Based Zero-Knowledge Proof Protocol for Privacy-Preserving Healthcare Data Sharing," *Preprint*, 2024.

[14] A. Mohamed, Y. Zhou, and L. Gao, "Leveraging Zero Knowledge Proofs for Blockchain-Based Identity Sharing in Healthcare," *Journal of Information Security and Applications*, vol. 76, 2024.

[15] T. Banerjee and P. Mishra, "A Combined Blockchain and Zero-Knowledge Model for Healthcare Data Privacy," *Journal of Cyber Policy*, vol. 8, no. 2, pp. 179–195, 2023.

[16] A. Halder and S. Saha, "Blockchain-Based Interoperable Healthcare Using Zero-Knowledge Proofs," in *Proc. Int. Conf. COMSNETS*, Bangalore, India, 2020, pp. 1–6.